Homework 2: PyTorch Basics

Total: 5 points

Overview

Practice tensor operations and automatic differentiation, then implement a complete MNIST classification workflow. You may use PyTorch, TorchVision, NumPy and the Python standard library. No GPU is required.

Questions 1–3: write your answers and include screenshots of your own execution results in the PDF report. Write any code needed for these questions yourself. For Question 1, include the execution result for 1.3; answer the remaining concept checks in writing.

Question 4: complete the marked TODOs in `HW2_PyTorch_Basics.ipynb`. This single programming question combines data loading, a two-layer MLP, loss computation, backpropagation, training, testing and weight serialization.

Use an isolated Python environment and select a compatible stable PyTorch/TorchVision pair at `https://pytorch.org/get-started/locally/`. Install Jupyter separately if needed. Record your Python, PyTorch and TorchVision versions and device. The scaffold uses seed 2026. Reproducibility across different releases and devices is not guaranteed.

1. Concept checks (1.00 point)

Each item is worth 0.25 point. In your PDF report, give an answer and a one- or two-sentence explanation. Run the code in 1.3 yourself and include a screenshot of the result; add printing statements as needed.

1.1. For class-index classification, `CrossEntropyLoss` receives logits of shape (N, C) and integer targets of shape $(N,)$. Should you apply `softmax` to the logits first? State the target dtype and valid label range.

1.2. Does `model.eval()` disable gradient computation? Explain how its purpose differs from `torch.no_grad()`.

1.3. In the code below, state `x.grad` and explain whether `y.grad` is populated by default. How would you retain the gradient of `y`?

```
x = torch.tensor(2.0, requires_grad=True)
y = 3 * x
loss = y ** 2
loss.backward()
```

1.4. If a classifier ends in softmax, is an MSE objective mathematically possible? Explain why cross-entropy is usually preferred for class-index classification. Distinguish possibility from recommended practice.

2. Tensor operations (1.00 point)

Write your own code. Include the operation expressions, explanations and screenshots of the printed results in your PDF report.

2.1. Arithmetic (0.50). Create float32 random tensors A, B of shape $(5, 5)$. Create C with `zeros_like(A)`. Compute and print $A + B$, $A - B$, the elementwise product and the matrix product. Label the last two results clearly. Do not use NumPy to perform these operations.

2.2. Shape and indexing (0.50). Create `T = torch.arange(24).reshape(6,4)`. Flatten it to $(24,)$ and extract rows 1 through 4 and columns 1 through 2, using zero-based indexing. The resulting central slice must have shape $(4, 2)$. Report both shapes and values. Explain the difference between `reshape` and `permute`.

3. Autograd and accumulation (0.75 point)

Write your own code. Include your analytical predictions, explanations and screenshots of all three gradient results in your PDF report. Let X be a float32 random tensor of shape $(3, 3)$ with `requires_grad=True`, and let

$$M = \begin{bmatrix} 1 & 0 & 2 \\ 1 & 1 & 0 \\ 0 & 3 & 1 \end{bmatrix}, \qquad L(X) = \sum_{i,j} X_{ij} M_{ij}.$$

The multiplication in this definition is **elementwise**.

(a) **Derivative (0.25).** Compute L, call `backward()`, print `X.grad`, and verify it against your analytical derivative.

(b) **Accumulation (0.25).** Recompute L with the same leaf tensor X and call `backward()` again without clearing gradients. Predict and verify the new `X.grad`. Do not reuse the already-freed forward graph.

(c) **Reset (0.25).** Set `X.grad = None`, recompute L, and verify the gradient after one more backward pass. Explain why a normal training step clears gradients.

4. MNIST classification with a two-layer MLP (2.00 points)

Complete only this question in the supplied notebook. Use the official MNIST training split (60,000 examples) for training and the official test split (10,000 examples) for the final evaluation. Keep seed 2026. Use CPU, batch size 64, SGD with learning rate 0.1, and 5 training epochs.

(a) **Data loading and preprocessing (0.50).** Use `torchvision.datasets.MNIST` and `torchvision.transforms.v2`. Convert each PIL image to a tensor, cast/scale it to float32 in $[0, 1]$, then apply `Normalize((0.5,), (0.5,))` to obtain values in $[-1, 1]$. Create both training and test datasets with the same transform. Create DataLoaders with batch size 64 and `num_workers=0`; shuffle the training set only. Print a training batch's shapes, dtypes and image range. Verify image shape $(64, 1, 28, 28)$ and label shape $(64,)$. Explain Dataset versus DataLoader and the meaning of `num_workers=0` in your report.

(b) **Model definition (0.25).** Define an `nn.Module` with exactly two fully connected layers:

$$(1, 28, 28) \xrightarrow{\text{flatten}} 784 \xrightarrow{\text{Linear}} 128 \xrightarrow{\text{ReLU}} 128 \xrightarrow{\text{Linear}} 10.$$

Here "two-layer" counts the two Linear layers: there is one hidden layer. Implement `forward` so the output has shape $(B, 10)$ and contains raw logits. Do not add an output softmax.

(c) **Loss, backpropagation and training (0.50).** Use `CrossEntropyLoss` and `SGD(lr=0.1)`. Each mini-batch step must clear gradients, compute logits, compute the loss against integer class labels,

call `backward()`, and update parameters. Use `model.train()` during training. Train for 5 epochs and report the sample-weighted mean training loss and training accuracy for each epoch. Also evaluate full-training-set loss before and after training using the evaluation function in part (d). Explain which line computes gradients and which line changes parameters.

(d) **Testing (0.25).** Implement an evaluation function using `model.eval()` and `torch.no_grad()`. Accumulate the loss weighted by batch size and the number of correct predictions; divide both by the total number of examples, including the smaller last batch. After all 5 epochs, report mean cross-entropy loss and classification accuracy on the entire test set. Do not train on the test set or use its results to choose hyperparameters or a checkpoint.

(e) **Saving and reloading (0.50).** Save the trained model's `state_dict` to `mnist_mlp.pt`. Recreate the same architecture and load with `map_location="cpu"` and `weights_only=True`. Put both models in evaluation mode and compare their logits on the same fixed image batch under `no_grad()`, using `torch.testing.assert_close`. Submit only a checkpoint produced by your own code.

The supplied `prepare_mnist.py` reuses the local compressed MNIST files and places them in the layout expected by TorchVision. The notebook calls this helper. Its `--source` argument accepts a different local data folder. If local data is unavailable, use `download=True`. The supplied student package includes local data.

5. Report (0.25 point)

Submit a PDF report with your name/student ID and environment information. Keep the presentation clear and the screenshots readable. Include:

- Written answers to Questions 1–3, the tensor operation expressions, the predicted gradients and explanations, and screenshots of your own execution results for 1.3, 2 and 3.
- Question 4's batch properties, Dataset/DataLoader explanation, per-epoch training loss/accuracy, initial/final full-training-set loss, final test loss/accuracy and reload check. Explain `backward()` versus `optimizer.step()`.

Training metrics accumulated during updates can differ from an evaluation of the final model on the training set. Label them clearly and report measurements from your own run.

Submission

Submit the completed Question 4 notebook with all cells executed, your PDF report and `mnist_mlp.pt`. Keep supporting code needed to reproduce your work, including your independently written code for Questions 1–3 and `prepare_mnist.py` if used. Do not submit dataset files or an environment directory. Cite external material and explain your own implementation.